

Алгоритмические языки

Экзамен за I семестр

1. Классы в языке Си ++ : объявление, поля и функции (методы) классов. Определение функций внутри класса и за пределами класса, примеры

Класс в языке C++ – это новый тип, определяемый программистом, включающий данные (поля класса) и методы (функции) для обработки этих данных. Переменные этого типа называются объектами

Класс – это структура, в которую введены методы для обработки полей. Объекты – это переменные типа класса

Формат объявления класса:

```
<Ключевое_слово> <Имя_класса> {  
    <список_компонент>  
};
```

В качестве ключевого слова используется одно из трёх ключевых слов:

1. struct
2. class
3. union

Поля и методы:

```
struct Complex {  
    double real, image; // Поля класса  
    void define(double re=0.0, double im=0.0) {  
        // Определение метода внутри класса, метод будет подставляемым  
        Real = re;  
        Image = im; // Обращение к полям внутри метода  
    }  
    void print(); // Описание метода  
};  
void Complex::print() { // Определение метода за пределами класса  
    printf("\nreal=%f image=%f", real, image);  
}
```

Обращение к полям и методам класса внутри методов класса просто по имени, а за пределами класса через имя объекта и операцию «.» или через имя указателя на объект и операцию «->». Каждый объект класса имеет в оперативной памяти свои копии полей класса

2. Статусы доступа компонент класса. Операции для доступа к компонентам класса, примеры

Доступность компонент класса. Свойство доступности определяет возможность доступа к полям и методам за пределами класса (через имя объекта или через указатель на объект). Существуют три статуса доступа:

1. `public` (полностью доступны за пределами класса);
2. `private` (не доступны за пределами класса, можно обращаться к компонентам только в методах своего класса);
3. `protected` (доступны только в своём классе и в производных классах).

По умолчанию, если класс определён с ключевым словом `struct`, то все компоненты имеют статус доступа `public`

Если с ключевым словом `union`, тоже `public`, но все поля каждого объекта располагаются в памяти, начиная с одного адреса

Если класс определён с ключевым словом `class`, то все поля и методы по умолчанию имеют статус доступа `private`

Статус доступа можно изменить с помощью соответствующих модификаторов:

1. `public:`
2. `private:`
3. `protected:`

Пример изменения статуса доступа:

```
struct A {  
    // Статус доступа public  
    private:  
        // Статус доступа private  
    protected:  
        // Статус доступа protected  
};  
  
class B {  
    // Статус доступа private  
    public:  
        // Статус доступа public  
    protected:  
        // Статус доступа protected  
};
```

3. Конструкторы и деструктор класса, конструктор копирования и конструктор перемещения, назначение, пример
--

Конструктор. Конструктор класса специальный блок операторов (инструкций), вызываемый при создании объекта. Назначение:

1. Присвоение начальных значений полям,
2. Выделение памяти,
3. Открытие файлов, сетевых соединений и т.п.

Имя конструктора совпадает с именем класса, конструктор не имеет возвращаемого значения. Возможна перегрузка конструкторов

Конструктор может определяться как внутри класса, так и за пределами. Формат определения конструктора внутри класса:

```
<Имя_класса> (<Список_формальных_параметров>) { <Операторы_тела конструктора> }
```

По умолчанию класс всегда имеет конструктор копирования вида:

```
A(const A& a) { ... } //(A – имя класса)
```

Если нет ни одного явного конструктора, то по умолчанию создаётся конструктор без параметров. Эти конструкторы можно переопределять.

Конструктор копирования по умолчанию можно удалить, для этого в классе заголовок конструктора объявляется с ключевым словом `delete` без тела, например:

```
A(const A& a) = delete;
```

Если нет явного конструктора копирования, то конструктор копирования по умолчанию будет удалён при наличии явного конструктора перемещения (см. ниже) или оператора копирования с перемещением

Примеры вызовов конструкторов:

```
A a1; A* pA=new A; // Вызываются конструкторы без параметров
A a2(3, 4); A * pA2=new A(3, 4); // Вызываются конструкторы с 2-мя параметрами
```

Ключевое слово `explicit` у конструктора с одним параметром запрещает это преобразование, допустима явная форма:

```
A a1(5);
A a2(a1);
```

Деструктор — специальный блок операторов (инструкций), служащий для деинициализации объекта (освобождение памяти, закрытие файлов и т.п.).

Вызывается автоматически при удалении объекта, например, оператором `delete` или при выходе из блока, в котором существует объект. Не имеет возвращаемого значения и параметров.

Может определяться как внутри класса, так и за пределами. Имеет следующий шаблон:

```
~<имя_класса>() {
    <тело_деструктора>
}
```

4. Статические компоненты класса, назначение, пример

Статические поля:

Память под обычные поля (нестатические) выделяется при создании объекта, каждый объект класса имеет свои копии обычных полей

Статические поля объявляются в классе с модификатором `static`, память под статические поля выделяется при определении класса, и не выделяется при создании объектов

Статические поля существуют в единственном экземпляре независимо от того, сколько объектов создано, то есть это поля класса, они общие для всех объектов. К статическим полям за пределами класса можно обращаться через имя объекта, как к обычным полям, но чаще к ним обращаются через имя класса (это можно делать даже когда объекты класса не созданы)

Статические поля обязательно требуют инициализации (за пределами класса)

Статические методы:

На статические поля распространяются модификаторы статуса доступа. Для доступа к собственным (`private`) или защищённым (`protected`) статическим полям за пределами класса служат открытые статические методы, которые определяются с модификатором `static`

К статическим методам за пределами класса обращаются как к статическим полям через имя объекта или чаще через имя класса

Внутри статических методов можно обращаться только к статическим полям или вызывать другие статические методы

К обычным полям и методам обращаться нельзя, так как при вызове статического метода объекты могут быть ещё не созданы (статический метод относится ко всему классу)

```
#include <iostream>
using namespace std;
class A {
    static int N; // Счётчик созданных объектов
public:
    A() {
        N++;
    }
    ~A() { N--; }
    static int getN();
};
int A::getN() {
    return N;
}
int A::N = 0; // Обязательная инициализация статического поля

int main(int argc, char* argv[]) {
    cout << "N = " << A::getN();
    A a1, a2, a3, a4;
    cout << "\nN = " << a1.getN();
    {
        A a5;
        cout << "\nN=" << a5.getN();
    }
    cout << "\nN=" << A::getN();
    return 0;
}
```

5. Указатель this, примеры использования

Иногда при вызове нестатического метода требуется обращаться к адресу объекта, для которого вызывается метод. Но когда программист пишет код метода класса, объектов ещё не существует

Для этих целей любой нестатический метод и конструкторы (деструктор) неявно (по умолчанию) получает указатель `this` – указатель на объект класса, для которого вызывается метод

Рассмотри два случая использования:

- Первый:

```
class point {
    int x,y;
public:
    point(int x, int y) {
        this->x = x;
        this->y = y;
    }
    ...
};
```

- Второй:

```
class List { // Линейный односвязный список
    int x; // инф. поле
    List *pNext; // Указатель на след. элемент public:
    List(int x) {
        this->x = x;
    }
    void add(List *&pF) { // Добавить элемент в начало списка
        pNext = pF; pF = this; // Текущий элемент будет первым
    }
    ...
};
```

6. Указатели на компоненты класса (на функции и поля класса), назначение, пример

Указатели на компоненты класса:

Операции `*` и `->` предназначены для работы с указателями на компоненты класса. До использования указателя его необходимо соответствующим образом настроить

Указатели на компонентные функции (методы):

1. Формат определения:

`<тип_возвр_значения> (<имя_класса>::*<имя_указателя>)(<спецификация_форм_параметров>);`

2. Настройка указателя:

`<имя_указателя>= &<имя_класса>::<имя_функции>;`

3. Вызов функции по указателю:

`(<имя_объекта>.*<имя_указателя>)(<параметры>);`

`(<имя_ук_на_об>->*<имя_указателя>)(<параметры>);`

Указатели на поля

1. Формат определения указателя на поле класса:

`<тип_поля> (<имя_класса>::*<имя_указателя>);`

2. Настройка указателя:

`<имя_указателя>= &<имя_класса>::<имя_поля>;`

3. Обращение к полю по указателю:

`<имя_объекта>.*<имя_указателя>=10; <имя_ук_на_об>->*<имя_указателя>=10;`

Рассмотрим пример:

```
#include <iostream>
using namespace std;
struct point {
    int x, y;
    void printX() {
        cout<<endl<<"x="<<x;
    }
    void printY() {
        cout<<endl<<"y="<<y;
    }
};
int main(int argc, char* argv[]) {
    void (point::*pF)()=&point::printX; // Указатель на функцию класса
    int point::*p=&point::x; // Указатель на поле класса
    point p1;// p1.x=5; // p1.y=10;
    p1.*p=5; // Обращение к полю x через указатель
    p=&point::y; // Указатель настраиваем на поле y
    p1.*p=10; // Обращение к полю y через указатель
    (p1.*pF)(); // Вызываем функцию printX через указатель
    pF=&point::printY; // Указатель на функцию класса настраиваем на функцию printY
    (p1.*pF)(); // Вызываем функцию printY через указатель
    return 0;
}
```

7. Дружественные функции и классы, назначение, пример

Дружественные функции класса. Особенности ДФ:

1. ДФ не получает указатель `this`, т.к. она не принадлежит классу
2. Дружественная функция получает доступ к объекту через формальный параметр, объект класса передаётся по указателю или по ссылке
3. Место размещение прототипа ДФ внутри класса безразлично, на неё не распространяется действие модификаторов доступа
4. ДФ может быть компонентной функцией другого класса
5. ДФ может быть дружественной по отношению к нескольким классам

Дружественные классы:

```
class C1 {  
    friend class C2;  
    ...  
};  
class C2 { ... }
```

Все функции класса C2 являются дружественными по отношению к классу C1

```
#include <iostream>  
using namespace std;  
class point {  
    int x, y;  
public:  
    point(int a, int b) {  
        x=a; y=b;  
    }  
    friend void print(point &p); // Заголовок ДФ внутри класса  
};  
void print(point &p) { // Дружественная функция  
    cout << endl<<"x="<<p.x<< " y=" << p.y; // Имеет доступ к закрытым полям (private)  
}  
int main(int argc, char* argv[]) {  
    point p1(3, 5); print(p1); // Вызов ДФ  
    return 0;  
}
```

8. Шаблоны классов в C ++ . Примеры использования

Шаблоны классов. Шаблон класса задаёт семейство классов, в которых могут быть использованы разные типы:

```
template <список_параметров_шаблона> <определение_шаблона_класса>
```

Имя параметра шаблона – это имя неизвестного заранее типа

При создании объекта указывается имя конкретного типа и по шаблону генерируется класс

Существует библиотека стандартных шаблонов (STL- Standard Template Library). В ней шаблонами задаются контейнеры – предназначенные для хранения набора объектов в памяти (объекты могут быть любых типов)

Контейнеры: vector, list, set, stack, queue

Рассмотрим пример:

```
#include <iostream>
using namespace std;
template <class T> { // Шаблон класса вектор (массив)
// T - тип элементов вектора
class vektor
    T* data; // Указатель на массив
    int size; // Размерность
    public:
    vektor(int n=10);
    ~vektor() { delete [] data; }
    T& operator[] (int i) { // Оператор функция возвращает элемент по индексу
        return data[i];
    }
    T getSum(); // Функция считает сумму элементов
};
// Внешнее определение конструктора template <class T>
// Перед любым определением метода за пределами класса
vektor<T>::vektor(int n) {
    data = new T[n];
    size = n;
}
// Внешнее определение функции template <class T>
// Перед любым определением метода за пределами класса
T vektor<T>::getSum() {
    T sum=0;
    for(int i = 0; i < size; i++)
        sum += data[i];
    return sum;
}
int main() {
    vektor<int> X(5); // Вектор элементов типа int
    X[1]=101;
    // Вызов оператор функции
    cout<<"x[1]="<< X[1];
    vektor<double> Y(10); // Создаётся класс вместо T подставляется
    double Y[0]=10.51;
    cout << "\nY[0] = " << Y[0];
    for(int i = 0; i < 10; i++) Y[i]=i;
    cout << "\nSum = "<< Y.getSum();
    return 0;
}
```

9. Перегрузка стандартных операций (унарных, бинарных) в случае, когда оператор – функция принадлежит классу, пример

Правило трёх (также известное как «Закон Большой Тройки» или «Большая Тройка») – правило в C++, гласящее, что если класс или структура определяет один из следующих методов, то они должны явным образом определить все три метода:

1. Деструктор
2. Конструктор копирования
3. Оператор присваивания копированием

С выходом одиннадцатого стандарта правило расширилось и теперь называется правило пяти. Теперь при реализации конструктора необходимо реализовать:

1. Деструктор
2. Конструктор копирования
3. Оператор присваивания копированием
4. Конструктор перемещения
5. Оператор присваивания перемещением

Это возможно, если хотя бы один из операндов является объектом класса, для этого создаётся специальная, так называемая, оператор – функция, которая может быть как членом класса, так и функцией, не принадлежащей классу.

Формат определения оператор – функции имеет вид:

```
<тип_возвращаемого_значения>
operator <знак_операции>
(спецификация_параметров) {
    операторы_тела_функции
}
```

Существует три способа перегрузки:

1. Оператор-функция определяется как функция, не принадлежащая классу;
2. Оператор-функция определяется как функция класса;
3. Оператор-функция определяется как дружественная функция класса.

Особенности перегрузки операций:

1. Можно перегружать только стандартные операции
2. Не допускают перегрузки операции: ‘.’, ‘.*’, ‘?:’, ‘::’, ‘sizeof’, ‘#’, ‘##’
3. При перегрузке сохраняется арность операций (унарная операция остаётся унарной, а бинарная – бинарной)
4. Бинарная операция перегружается либо как функция, не принадлежащая классу с двумя параметрами, один обязательно объект (ссылка на объект) класса, или как функция класса с одним параметром, первым операндом операции выступает объект класса, для которого вызывается функция
5. Бинарные операции ‘=’, ‘[]’, ‘->’ должны обязательно определяться как компонентные функции класса
6. Унарная операция перегружается либо как функция, не принадлежащая классу с одним параметром
7. Объектом (ссылкой на объект) класса, или как функция класса без параметров, операндом операции выступает объект класса, для которого вызывается функция

Так как мы не можем привести пример всевозможных перегрузок, оставим это как домашнее задание

10. Перегрузка стандартных операций (унарных, бинарных) в случае, когда оператор – функция не принадлежит классу, пример

Правило трёх (также известное как «Закон Большой Тройки» или «Большая Тройка») – правило в C++, гласящее, что если класс или структура определяет один из следующих методов, то они должны явным образом определить все три метода:

1. Деструктор
2. Конструктор копирования
3. Оператор присваивания копированием

С выходом одиннадцатого стандарта правило расширилось и теперь называется правило пяти. Теперь при реализации конструктора необходимо реализовать:

1. Деструктор
2. Конструктор копирования
3. Оператор присваивания копированием
4. Конструктор перемещения
5. Оператор присваивания перемещением

Это возможно, если хотя бы один из операндов является объектом класса, для этого создаётся специальная, так называемая, оператор – функция, которая может быть как членом класса, так и функцией, не принадлежащей классу.

Формат определения оператор – функции имеет вид:

```
<тип_возвращаемого_значения>
operator <знак_операции>
(спецификация_параметров) {
    операторы_тела_функции
}
```

Существует три способа перегрузки:

1. Оператор-функция определяется как функция, не принадлежащая классу;
2. Оператор-функция определяется как функция класса;
3. Оператор-функция определяется как дружественная функция класса.

Особенности перегрузки операций:

1. Можно перегружать только стандартные операции
2. Не допускают перегрузки операции: ‘.’, ‘.*’, ‘?:’, ‘::’, ‘sizeof’, ‘#’, ‘##’
3. При перегрузке сохраняется арность операций (унарная операция остаётся унарной, а бинарная – бинарной)
4. Бинарная операция перегружается либо как функция, не принадлежащая классу с двумя параметрами, один обязательно объект (ссылка на объект) класса, или как функция класса с одним параметром, первым операндом операции выступает объект класса, для которого вызывается функция
5. Бинарные операции ‘=’, ‘[]’, ‘->’ должны обязательно определяться как компонентные функции класса
6. Унарная операция перегружается либо как функция, не принадлежащая классу с одним параметром
7. Объектом (ссылкой на объект) класса, или как функция класса без параметров, операндом операции выступает объект класса, для которого вызывается функция

Так как мы не можем привести пример всевозможных перегрузок, оставим это как домашнее задание

11. Особенности перегрузки операций ++ и – в префиксной и постфиксной формах

Правило трёх (также известное как «Закон Большой Тройки» или «Большая Тройка») – правило в C++, гласящее, что если класс или структура определяет один из следующих методов, то они должны явным образом определить все три метода:

1. Деструктор
2. Конструктор копирования
3. Оператор присваивания копированием

С выходом одиннадцатого стандарта правило расширилось и теперь называется правило пяти. Теперь при реализации конструктора необходимо реализовать:

1. Деструктор
2. Конструктор копирования
3. Оператор присваивания копированием
4. Конструктор перемещения
5. Оператор присваивания перемещением

Перегрузка операций ++ и --, чтобы отличить постфиксную форму от префиксной, вводится формальный параметр типа int

Пример объявления операторов-функций внутри класса:

```
Vector & operator ++() // Префиксная форма операции ++
Vector operator ++(int) // Постфиксная форма операции ++
                        // всегда есть формальный (фиктивный)
                        // параметр типа int
```

Пример объявления операторов-функций за пределами класса:

```
Vector & operator ++(Vector & V) // Префиксная форма операции ++
Vector operator ++(Vector & V, int) // Постфиксная форма операции ++
                                   // всегда есть формальный
                                   // (фиктивный) параметр типа int
```

В постфиксной форме вначале с помощью конструктора копирования создаём новый объект, в котором сохраняется начальное (старое) состояние текущего объекта, затем текущий объект изменяется, и возвращается созданный объект

12. Особенности перемещения объекта, продемонстрировать пример перемещения при перегрузке операции « = » (присваивание) на примере своего класса вектор, содержащем в качестве поля динамический массив элементов типа `int`

Правило трёх (также известное как «Закон Большой Тройки» или «Большая Тройка») – правило в C++, гласящее, что если класс или структура определяет один из следующих методов, то они должны явным образом определить все три метода:

1. Деструктор
2. Конструктор копирования
3. Оператор присваивания копированием

С выходом одиннадцатого стандарта правило расширилось и теперь называется правило пяти. Теперь при реализации конструктора необходимо реализовать:

1. Деструктор
2. Конструктор копирования
3. Оператор присваивания копированием
4. Конструктор перемещения
5. Оператор присваивания перемещением

По аналогии с конструкторами копирования и перемещения по умолчанию существуют в классе **операторы присваивания с копированием и присваивания с перемещением**

Правила создания и использования аналогичные. Эти операторы можно переопределять или удалять, например:

```
A & operator =(const A & Ob) = delete;
```

```
A & operator =(A && Ob) = delete;
```

13. Наследование классов. Объявление производного класса, пример использования

Общие сведения о наследовании. Основная идея:

На базе существующего класса (класс родитель или базовый класс), создаётся производный класс (класс – наследник, дочерний класс), который включает в себя поля и функции базового класса (наследует их), и содержит дополнительные поля (обладает новыми свойствами) и функции:

```
class S: X, Y, Z { ... } ;  
class B: public A { ... };  
class D: public X, protected B { ... };
```

Особенности конструкторов при наследовании:

1. Конструктор производного класса в первую очередь всегда должен вызывать конструктор базового класса
2. Если это действие не выполняется явно, то по умолчанию вызывается конструктор без параметров (если он есть, если его нет, будет ошибка)
3. Если класс имеет несколько базовых, то конструкторы базовых классов должны вызываться в порядке перечисления этих классов в списке базовых

Особенности деструкторов при наследовании:

1. Деструктор производного класса всегда неявно по умолчанию после выполнения своего тела вызывает деструкторы базовых классов. Причём порядок разрушения объекта (вызовов деструкторов) обратен порядку создания (вызова конструкторов)

14. Статусы доступа при наследовании классов

Перед именем базового класса может указываться **статус доступа** наследования или тип наследования доступа (одно из ключевых слов `public`, `protected`, `private`):

1. Статус доступа наследования определяет статус доступа наследуемых полей и функций из базового класса внутри производного класса
2. Производный класс может определяться с ключевым словом `struct` или `class` (с ключевым словом `union` производный класс не определяется)
3. Если производный класс определён с ключевым словом `class`, то по умолчанию статус доступа наследования `private`
4. Если производный класс определён с ключевым словом `struct`, то по умолчанию статус доступа наследования `public`

Наследование	Доступ в базовом классе	Доступ в производном классе
<code>public</code>	<code>public</code> <code>protected</code> <code>private</code>	<code>public</code> <code>protected</code> <code>private</code>
<code>protected</code>	<code>public</code> <code>protected</code> <code>private</code>	<code>protected</code> <code>protected</code> <code>private</code>
<code>private</code>	<code>public</code> <code>protected</code> <code>private</code>	<code>private</code> <code>private</code> <code>private</code>

15. Конструкторы и деструктор в производных классах, примеры

Начнём с того, что когда мы создаём элементы (переменные) класса, мы не можем присвоить им значения в самом определении класса. Компилятор выдаст ошибку. Поэтому нам необходимо создавать отдельный метод (так называемую set-функцию) класса, с помощью которого и будет происходить инициализация элементов

Конструктор (от слова construct — создавать) — это специальный метод класса, который предназначен для инициализации элементов класса некоторыми начальными значениями

В отличие от конструктора, **деструктор** (от слова destruct — разрушать) — специальный метод класса, который служит для уничтожения элементов класса. Чаще всего его используют тогда, когда в конструкторе, при создании объекта класса, динамически был выделен участок памяти и необходимо эту память очистить, если эти значения уже не нужны для дальнейшей работы программы

Важно запомнить:

1. Конструктор и деструктор, мы всегда объявляем в разделе public;
2. При объявлении конструктора, тип данных возвращаемого значения не указывается, в том числе — void
3. У деструктора также нет типа данных для возвращаемого значения, к тому же деструктору нельзя передавать никаких параметров
4. Имя класса и конструктора должно быть идентично; Имя деструктора идентично имени конструктора, но с приставкой ~
5. В классе допустимо создавать несколько конструкторов, если это необходимо. Имена, согласно пункту 2 нашего списка, будут одинаковыми. Компилятор будет их различать по передаваемым параметрам (как при перегрузке функций)
6. Если мы не передаём в конструктор параметры, он считается конструктором по умолчанию
7. Обратите внимание на то, что в классе может быть объявлен только один деструктор:

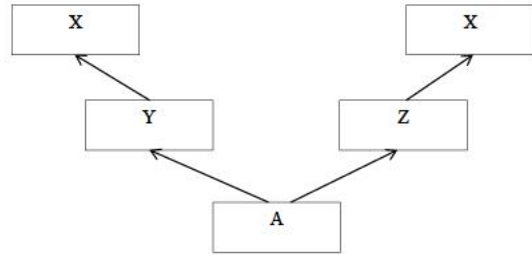
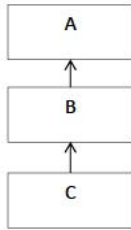
```
# include <iostream>
using namespace std;
class AB {
    int a, b;
public:
    AB() {
        a = 0; b = 0;
        cout << "Работа конструктора при создании нового объекта: " << endl;
        cout << "a = " << a << endl;
        cout << "b = " << b << endl << endl;
    }
    void setAB() {
        cout << "Введите целое число a: ";
        cin >> a;
        cout << "Введите целое число b: ";
        cin >> b;
    }
    void getAB() { //выведем на экран изменённые значения
        cout << "a = " << a << endl;
        cout << "b = " << b << endl << endl;
    }
};

int main() {
    AB obj1;
    obj1.setAB(); //присвоим новые значения переменным
    obj1.getAB(); //и выведем их на экран
    AB obj2;
    return 0;
}
```

16. Множественное наследование и виртуальные базовые классы, примеры

Множественное наследование и виртуальные базовые классы

Представление отношения наследования в виде диаграммы классов:



Пример заголовков классов:

```
class X { ... public: int k; void f() { ... } ... };
class Y: public X { ... } ;
class Z: public X { ... };
class A: public Y, public Z { ... };
```

При такой схеме наследования происходит дублирование полей и функций класса X в классе A:

```
A d;
d.A::Y::X::k=5;
d.A::Y::X::f();
d.A::Z::X::k=10;
d.A::Z::X::f();
```

```
-----
d.Y::k=5;
d.Y::f();
d.Z::k=10;
d.Z::f();
```

```
-----Внутри A-----
Y::X::k=5; Y::X::f();
Z::X::k=5; Z::X::f();
```

```
-----
Y::k=5; Y::f();
Z::k=5; Z::f();
```

Для устранения возможность дублирования полей и методов, когда они наследуются из 1-го класса при множественном наследовании, введено понятие виртуального класса, в предыдущем примере класс X должен быть объявлен как виртуальный, для этого используется ключевое слово `virtual`

Виртуальный класс — класс, который при множественном наследовании дополнительно включается в классы-потомки ссылкой на объект этого класса во избежания дублирования.

Для обычного наследования в C++ в оперативной памяти в объекте производного класса вначале идут поля базового класса, а затем поля производного, при виртуальном наследовании дополнительно появляется ссылка в производном классе

Термин «виртуальный» собственно относится не к базовому классу, а к способу наследования от него

17. Переопределение функций при наследовании классов, виртуальные функции в Си ++ (таблица виртуальных функций). Статическое и динамическое связывание. Примеры

Переопределение функций. При наследовании классов можно переопределить функции базового класса в производном классе. Этот процесс называется переопределением функций. Для этого достаточно просто определить в производном классе функцию с тем же именем и типом возвращаемого значения, что и у функции базового класса

Виртуальные функции позволяют более эффективно работать с наследованием и переопределением функций. Это означает, что при вызове виртуальной функции будет выполнен код соответствующей функции производного класса вместо функции базового класса. Таким образом, программист может вызвать функцию подходящего производного класса в соответствии с объектом, на котором выполняется функция

Динамическое связывание. С++ использует таблицу виртуальных функций для определения, какая функция будет вызвана. Эта таблица хранит указатели на виртуальные функции, определённые для каждого класса в иерархии наследования. Когда вызывается виртуальная функция, программа ищет таблицу виртуальных функций и находит указатель на функцию, соответствующую вызываемому объекту. Затем программа вызывает функцию по найденному указателю

Статическое связывание происходит во время компиляции, когда все символы программы уже известны. В этом случае компилятор выбирает правильную функцию на основе типа переменной, которая ссылается на объект. Динамическое связывание происходит во время выполнения программы, когда тип объекта не известен на момент компиляции и определяется во время выполнения

```
#include <iostream>
using namespace std;

class Base {
public:
    virtual void print() {
        cout << "Base Class" << endl;
    }
};

class Derived : public Base {
public:
    void print() {
        cout << "Derived Class" << endl;
    }
};

int main() {
    Base* bptr = new Base();
    bptr->print();

    Derived* dptr = new Derived();
    dptr->print();

    Base* bptr2 = new Derived();
    bptr2->print();

    delete bptr;
    delete dptr;
    delete bptr2;

    return 0;
}
```

18. Абстрактные и локальные классы в C ++ , примеры

Абстрактные классы - это классы, которые содержат или наследуют без переопределения хотя бы одну чистую виртуальную функцию. Абстрактный класс определяет интерфейс для переопределения производными классами

Чистые виртуальные функции - это функции, которые не имеют определения. Чтобы определить виртуальную функцию как чистую, её объявление завершается значением "= 0". Например, определим абстрактный класс, который представляет геометрическую фигуру:

```
class Figure {  
public:  
    virtual double getSquare() = 0;  
    virtual double getPerimeter() = 0;  
    virtual void showFigureType() = 0;  
};
```

Класс Figure является абстрактным, потому он содержит как минимум одну чистую виртуальную функцию. А в данном случае даже три таких функции. И ни одна из функций не имеет никакой реализации. Реализацию должны определять классы-наследники. Класс, объявленный внутри функции, становится локальным для этой функции и называется локальным классом в C++

Имя локального класса может использоваться только локально, то есть внутри функции, а не вне её. Методы локального класса должны быть определены только внутри него. Локальный класс может иметь статические функции, но не статические элементы данных

Например, в следующей программе Test является локальным классом в fun():

```
#include <iostream>  
using namespace std;  
void fun() {  
    class Test {  
        ...  
    };  
}  
int main() {  
    return 0;  
}
```

19. Требования к классам, объекты которых являются элементами последовательных контейнеров. Лямбда – функции в C++. Использование лямбда – функций в алгоритмах. Сортировка контейнера алгоритмом `sort` с помощью предиката, лямбда – функции и перегрузки операции «`<`»

Правило трёх (также известное как «Закон Большой Тройки» или «Большая Тройка») – правило в C++, гласящее, что если класс или структура определяет один из следующих методов, то они должны явным образом определить все три метода:

1. Деструктор
2. Конструктор копирования
3. Оператор присваивания копированием

С выходом одиннадцатого стандарта правило расширилось и теперь называется правило пяти. Теперь при реализации конструктора необходимо реализовать:

1. Деструктор
2. Конструктор копирования
3. Оператор присваивания копированием
4. Конструктор перемещения
5. Оператор присваивания перемещением

Классы, объекты которых являются элементами последовательных контейнеров, должны иметь конструктор по умолчанию и операторы сравнения (оператор «`<<`»). Оператор «`<<`» используется при сортировке контейнеров и для упорядочивания элементов в контейнере

Лямбда-функция – это функция без имени, которую можно определить внутри другой функции или передать как параметр в другую функцию. Лямбда-функции часто используются вместе с алгоритмами STL для выполнения операций на коллекциях

Пример использования лямбда-функции в алгоритме `for_each`:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> vec = { 1, 3, 5, 7 };

    for_each(vec.begin(), vec.end(), [](int n) {
        cout << n << " ";
    });

    return 0;
}
```

Пример сортировки контейнера вектора с помощью предиката:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

bool predicate(int a, int b) {
    return a > b;
}

int main() {
    vector<int> vec = { 1, 3, 5, 7 };
    sort(vec.begin(), vec.end(), predicate);

    for (int n : vec) {
        cout << n << " ";
    }

    return 0;
}
```

Пример сортировки контейнера вектора с помощью лямбда-функции:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> vec = { 1, 3, 5, 7 };
    sort(vec.begin(), vec.end(), [](int a, int b) {
        return a > b;
    });

    for (int n : vec) {
        cout << n << " ";
    }

    return 0;
}
```

20. Функциональные объекты (функторы), перегрузка операции () (вызов функции) в классах, пример использования

Функциональный объект (функтор) – это объект, который можно использовать как функцию. В отличие от обычных функций, функциональные объекты могут хранить состояние и данные, а также могут передаваться в качестве параметров в другие функции

Перегрузка операции () (вызов функции) позволяет создать функцию-объект, которая может вызываться как обычная функция. Функция-объект может быть создана с помощью класса, который содержит перегруженный оператор ()

Пример:

```
#include <iostream>
using namespace std;

class Sum {
public:
    int operator() (int a, int b) {
        return a + b;
    }
};

int main() {
    Sum s;
    cout << s(2, 3) << endl;

    return 0;
}
```

21. Требования к классам, объекты которых являются элементами контейнеров set и map

Классы, объекты которых являются элементами контейнеров set и map, должны иметь конструктор по умолчанию и операторы сравнения (оператор «<») для упорядочивания элементов в контейнере. Кроме того, для использования объекта в качестве ключа в контейнере map этот объект также должен иметь определённый оператор «==»

Классы, объекты которых используются в качестве ключей в контейнере set и map, также должны иметь работающий хэш-код, который может быть реализован с использованием функции hash.

22. Требования к классам, объекты которых являются элементами контейнеров unordered_set и unordered_map

Классы, объекты которых могут быть элементами контейнеров unordered_set и unordered_map, должны удовлетворять определенным требованиям:

1. Должен быть определён оператор равенства (==), который определяет равенство двух объектов этого класса
2. Должен быть определён хэш-код (оператор std::hash), который преобразует объект класса в значение типа size_t

23. Понятие особой (исключительной) ситуации в C++. Общий формат обработки исключительных ситуаций (ключевые слова: try, catch, throw)

Исключение – это специальный механизм обработки ошибок в C++. Оно позволяет выйти из функции, если в ней произошла какая-либо ошибка, и передать обработку ошибки в вызывающую функцию или в программу в целом.

Синтаксис обработки исключений в C++:

```
try {<код, который может вызвать исключение>}  
  
catch (<тип_исключения исключение>) {<код обработки исключения>}
```

24. Обработка исключения внутри функции, в которой сгенерировано исключение (ключевые слова: try, catch, throw), пример

Внутри функции, в которой произошла ошибка, можно генерировать исключения и обрабатывать их в этой же функции:

```
void do_something() {  
    int a = 5;  
    int b = 0;  
    try {  
        if (b == 0) {  
            throw std::runtime_error("Division by zero");  
        }  
        int result = a / b;  
        std::cout << result << std::endl;  
    } catch (std::exception& e) {  
        std::cout << "Error: " << e.what() << std::endl;  
    }  
}
```

25. Обработка исключения вне функции, в которой сгенерировано исключение, объявление функции, в которой генерируются исключения (ключевые слова: try, catch, throw), пример

Если исключение произошло внутри функции, но обработать его нужно в вызывающей функции или в программе в целом, то можно объявить функцию, которая генерирует исключения:

```
void foo() {  
    throw std::runtime_error("Oops");  
}  
void bar() {  
    try {  
        foo();  
    } catch (std::exception& e) {  
        std::cout << "Error: " << e.what() << std::endl;  
    }  
}
```

26. Создание исключения как объекта класса в C++ (ключевые слова: try, catch, throw), пример

Исключение можно создать как объект класса и передать его в блок catch:

```
class MyException : public std::exception {
public:
    const char* what() const noexcept override {
        return "My exception occurred";
    }
};

void foo() {
    throw MyException();
}

void bar() {
    try {
        foo();
    } catch (MyException& e) {
        std::cout << "Error: " << e.what() << std::endl;
    }
}
```

27. Формы обработчиков исключений (catch), примеры

Обработка исключений – это механизм, позволяющий обработать ошибки, возникающие во время выполнения программы. Обработка исключений в C++ осуществляется с помощью конструкции try-catch. В блоке try помещается код, который может порождать ошибки, а в блоке catch помещается код, который будет обрабатывать исключения

Синтаксис конструкции try-catch выглядит следующим образом:

```
try {  
    // код, который может порождать исключения  
}  
  
catch (тип_исключения1 переменная1) {  
    // обработка исключения типа тип_исключения1, с параметром переменная1  
}  
  
catch (тип_исключения2 переменная2) {  
    // обработка исключения типа тип_исключения2, с параметром переменная2  
}  
  
...  
  
catch (тип_исключенияN переменнаяN) {  
    // обработка исключения типа тип_исключенияN, с параметром переменнаяN  
}
```

Если в блоке try произойдет исключение указанного типа, то управление передаётся в соответствующий блок catch, где может быть выполнен код обработки исключения.

Пример:

```
#include <iostream>  
using namespace std;  
  
int main() {  
    try {  
        int a, b;  
        cout << "Введите числа для деления:" << endl;  
        cin >> a >> b;  
  
        if (b == 0) {  
            throw "Деление на ноль недопустимо!";  
        }  
        cout << "Результат деления: " << a / b << endl;  
    }  
    catch (const char* msg) {  
        cerr << "Ошибка: " << msg << endl;  
    }  
    return 0;  
}
```

В этом примере в блоке try пользователь вводит два числа и выполняется деление первого числа на второе. Если пользователя введёт 0 вместо второго числа, то будет выброшено исключение типа const char*, а затем выведено сообщение об ошибке

28. Формы выражений генерации исключений в C++ (throw), примеры

Оператор throw используется для генерации исключения в C++. Он позволяет явно указать, что произошла ошибка, и передать информацию об этой ошибке обработчику исключений

Формы выражений генерации исключений:

1. throw expression;
2. throw type();
3. throw;

1. Выражение expression может быть любым выражением языка C++. Оно служит для передачи информации об ошибке обработчику исключений

Пример:

```
#include <iostream>

class DivideByZeroException {
public:
    DivideByZeroException() {}
};

int divide(int x, int y) {
    if (y == 0) {
        throw DivideByZeroException();
    }
    return x / y;
}

int main() {
    int x = 10, y = 0;
    try {
        int z = divide(x, y);
        std::cout << "z = " << z << std::endl;
    }
    catch (DivideByZeroException& e) {
        std::cerr << "Ошибка: деление на ноль!" << std::endl;
    }
}
```

2. Форма `throw type()` используется для генерации исключения без передачи каких-либо дополнительных данных об ошибке

Пример:

```
#include <iostream>

class MyException {
public:
    MyException() {}
};

int main() {
    try {
        throw MyException();
    }
    catch (MyException& e) {
        std::cerr << "Ошибка!" << std::endl;
    }
}
```

3. Форма `throw` может использоваться для проброса уже обработанного исключения дальше по стеку вызовов

Пример:

```
#include <iostream>

int main() {
    try {
        try {
            throw "Ошибка 1!";
        } catch (const char* msg) {
            std::cerr << "Ошибка: " << msg << std::endl;
            throw;
        }
    } catch (const char* msg) {
        std::cerr << "Проброшено исключение: " << msg << std::endl;
    }
}
```

29. Обработка исключений в стандартной библиотеке C++ : обзор основных классов

Стандартная библиотека C++ содержит набор классов и функций для обработки исключений. Основные классы для обработки исключений в стандартной библиотеке C++:

1. `std::exception` - базовый класс для всех стандартных исключений в C++. Он содержит виртуальный метод `what()` для получения описания исключения
2. `std::logic_error` - класс для исключений, связанных с ошибками логики программы, например, исключение `invalid_argument`. Он также содержит конструкторы для создания объектов исключений с параметрами
3. `std::runtime_error` - класс для исключений, связанных с ошибками времени выполнения, например, исключение `out_of_range`. Он также содержит конструкторы для создания объектов исключений с параметрами

Пример:

```
#include <iostream>
#include <stdexcept>

int main() {
    try {
        int a, b;
        std::cout << "Введите числа для деления:" << std::endl;
        std::cin >> a >> b;

        if (b == 0) {
            throw std::runtime_error("Деление на ноль недопустимо!");
        }
        std::cout << "Результат деления: " << a / b << std::endl;
    } catch (std::exception& e) {
        std::cerr << "Ошибка: " << e.what() << std::endl;
    }
    return 0;
}
```

В этом примере используется класс `std::runtime_error` для выбрасывания исключения при делении на 0. Обрабатывается оно в блоке `catch` типа `std::exception`, где вызывается метод `what()` для получения описания ошибки

30. Обработка исключений в стандартной библиотеке C++: создание своего класса исключения на основе класса `std::exception`, пример

Для создания своего собственного класса исключения необходимо унаследоваться от класса `std::exception`. В наследуемом классе необходимо определить конструкторы для создания объектов исключений с параметрами, а также переопределить виртуальный метод `what()` для получения описания ошибки

Пример:

```
#include <iostream>
#include <exception>

class MyException: public std::exception {
public:
    const char* what() const noexcept override {
        return "Мое исключение!";
    }
};

int main() {
    try {
        throw MyException();
    } catch (std::exception& e) {
        std::cerr << "Ошибка: " << e.what() << std::endl;
    }
    return 0;
}
```

В этом примере создаётся класс исключения `MyException`, унаследованный от класса `std::exception`, переопределяющий метод `what()`. Этот класс используется для выброса исключения и обработки его в блоке `catch` типа `std::exception`

**31. Многозадачность в стандартной библиотеке C++ : высокоуровневый интерфейс
(функция `std::async()` и шаблон класса `std::future<>`),
пример создания потока и получение результата потоковой функции**

Стандартная библиотека C++ содержит функцию `std::async()` и шаблон класса `std::future<>` для создания и выполнения задач в отдельных потоках

Функция `std::async()` создает асинхронную задачу и возвращает объект типа `std::future<>`, который содержит результат выполнения задачи. Чтобы получить результат выполнения задачи, необходимо вызвать метод `get()` у объекта `std::future<>`

Пример:

```
#include <iostream>
#include <future>

double calculation(int x, double y) {
    return x * y;
}

int main() {
    int x = 10;
    double y = 5.5;
    std::future<double> result = std::async(std::launch::async, calculation, x, y);
    std::cout << "Результат: " << result.get() << std::endl;
    return 0;
}
```

В этом примере функция `calculation()` выполняется в отдельном потоке с помощью функции `std::async()`. В результате выполнения функции создаётся объект `std::future<>`, содержащий результат выполнения функции. Результат можно получить с помощью метода `get()` этого объекта

32. Многозадачность в стандартной библиотеке C++ : низкоуровневый интерфейс (использование класса `std::thread`), пример

Создание и выполнение задач в потоках также возможно с использованием класса `std::thread`

Класс `std::thread` представляет собой поток выполнения кода. Для запуска отдельного потока следует создать объект класса `std::thread`, передав ему функцию, которую нужно выполнить в отдельном потоке

Пример:

```
#include <iostream>
#include <thread>

void print(int x) {
    std::cout << "Выполняется функция в потоке " << std::this_thread::get_id() <<
        std::endl;
    std::cout << "x = " << x << std::endl;
}

int main() {
    std::thread t1(print, 100);
    std::thread t2(print, 200);

    std::cout << "Выполняется функция в основном потоке " << std::this_thread::get_id()
        << std::endl;

    t1.join();
    t2.join();

    return 0;
}
```

В этом примере создаются два объекта класса `std::thread`, каждый из которых запускает функцию `print()` в отдельном потоке. Программа продолжает работу в основном потоке. Чтобы дождаться завершения выполнения потоков, используется метод `join()` объектов `std::thread`

33. Синхронизация в стандартной библиотеке C++. Использование класса `std::mutex` (взаимное исключение), пример

Для синхронизации доступа к общим ресурсам в стандартной библиотеке C++ используется класс `std::mutex`

Класс `std::mutex` позволяет создать мьютекс – объект, который обеспечивает взаимное исключение при доступе к общему ресурсу. При использовании мьютекса доступ к общему ресурсу блокируется до тех пор, пока другой поток не освободит мьютекс

Пример:

```
#include <iostream>
#include <vector>
#include <mutex>
#include <thread>

std::mutex mtx;

void add(std::vector<int>& v, int n) {
    mtx.lock();
    for (int i = 0; i < n; i++) {
        v.push_back(i);
    }
    mtx.unlock();
}

int main() {
    std::vector<int> v1, v2;

    std::thread t1(add, std::ref(v1), 1000);
    std::thread t2(add, std::ref(v2), 1000);

    t1.join();
    t2.join();

    std::cout << "v1.size() = " << v1.size() << std::endl;
    std::cout << "v2.size() = " << v2.size() << std::endl;

    return 0;
}
```

В этом примере два потока выполняют функцию `add()`, которая добавляет `n` элементов в вектор. Для обеспечения взаимного исключения при доступе к вектору используется объект `mtx` класса `std::mutex`. Вызовы методов `lock()` и `unlock()` осуществляют блокировку и разблокировку мьютекса при доступе к общему ресурсу

34. Использование для синхронизации потоков блокировок – шаблона `std::lock_guard`, пример

Шаблон `std::lock_guard` используется для предотвращения состояния гонки при работе с общими ресурсами в многопоточной среде. Он блокирует указанные мьютексы, а затем автоматически освобождает их при выходе из блока

Пример использования:

```
#include <mutex>
#include <thread>

std::mutex mtx;

void func() {
    std::lock_guard<std::mutex> lock(mtx);
    // Критическая секция
    // ...
}
```

35. Использование условных переменных (объекты класса `std::condition_variable`) для синхронизации потоков в стандартной библиотеке C++: посылка и принятие оповещений потоками, примеры

Объекты класса `std::condition_variable` позволяют потокам ожидать событий, которые будут произведены другими потоками. Они позволяют потоку освободить процессор и не потреблять ресурсы, пока не наступят определённые условия

Пример использования:

```
#include <mutex>
#include <condition_variable>
#include <thread>

std::mutex mtx;
std::condition_variable cv;

bool dataReady = false;

void producer() {
    // ... {
        std::lock_guard<std::mutex> lock(mtx); // Захватываем мьютекс
        dataReady = true; // Устанавливаем признак готовности данных
    } // Мьютекс автоматически освобождается при выходе из блока
    cv.notify_one(); // Оповещаем один из ожидающих потоков
}

void consumer() {
    std::unique_lock<std::mutex> lock(mtx); // Захватываем мьютекс
    cv.wait(lock, [](){ return dataReady; }); // Ожидаем признак готовности данных
    // Критическая секция
    // ...
}
```

36. Использование атомарных операций (шаблон `std::atomic`) для синхронизации потоков в стандартной библиотеке C++

Шаблон `std::atomic` используется для выполнения атомарных операций с данными, что позволяет избежать состояния гонки

Пример использования:

```
#include <atomic>
#include <thread>

std::atomic<int> counter;

void func() {
    counter++; // Атомарное увеличение
    // ...
}
```

37. Умный указатель (smart pointer) `std::unique_ptr`, внутреннее устройство, пример использования

Умный указатель `std::unique_ptr` используется для хранения указателя на объект, который будет освобождён при уничтожении указателя. При копировании или присваивании указателя его ресурс переходит к новому объекту, а старый указатель становится недействительным

Пример использования:

```
#include <memory>

void func() {
    std::unique_ptr<int> ptr(new int(10)); // Создаём объект
    // Используем ptr
} // Объект автоматически удаляется при выходе из блока
```

38. Умный указатель (smart pointer) `std::shared_ptr`, внутреннее устройство, пример использования

Умный указатель `std::shared_ptr` используется для хранения указателя на объект, который имеет общее владение несколькими указателями. При копировании или присваивании указателя счётчик ссылок увеличивается, а при уничтожении указателя он уменьшается. Объект удаляется, когда его счётчик ссылок становится равным нулю

Пример использования:

```
#include <memory>

void func() {
    std::shared_ptr<int> ptr1(new int(10)); // Создаём объект
    std::shared_ptr<int> ptr2 = ptr1; // Объект имеет два указателя
    // Используем ptr1 и ptr2
} // Объект удаляется при выходе из блока, когда его счётчик ссылок становится равным нулю
```

39. Умный указатель (smart pointer) std::weak_ptr, назначение, пример использования

Умный указатель std::weak_ptr используется для хранения слабой ссылки на объект, который имеет общее владение несколькими указателями. Он не увеличивает счётчик ссылок и может быть преобразован в shared_ptr при наличии валидного указателя

Пример использования:

```
#include <memory>

void func() {
    std::shared_ptr<int> ptr(new int(10)); // Создаём объект
    std::weak_ptr<int> wp = ptr; // Создаём слабую ссылку
    // Используем wp
    if (auto sp = wp.lock()) { // Преобразуем в shared_ptr
        // Используем sp
    }
} // Объект удаляется при выходе из блока, когда его счётчик ссылок становится равным нулю
```

40. Перегрузка операторов new и delete

Оператор new используется для выделения динамической памяти, в то время как оператор delete – для освобождения этой памяти. Эти операторы можно перегружать для управления процессом выделения и освобождения памяти

В частности, оператор new можно перегружать для реализации собственного аллокатора памяти, который будет использоваться вместо стандартного оператора new. Такой аллокатор может позволить ускорить выделение памяти для определённых типов данных или ограничить количество доступной памяти

Оператор delete также можно перегружать, чтобы освободить дополнительные ресурсы, выделенные при выделении динамической памяти

41. Размещающий оператор new и его перегрузка

Размещающий (placement) оператор new принимает параметры, указывающие, где в памяти следует разместить объект. Такие операторы могут быть использованы для сохранения состояния объектов при изменении адреса их расположения в памяти

Перегрузка размещающего оператора new позволяет специфицировать условия выделения памяти и размещения объектов, что может быть полезным в приложениях реального времени и при использовании многопоточности

42. Передача дополнительных параметров в оператор new при его перегрузке

Значения **дополнительных параметров** могут передаваться в определённую перегруженную версию оператора new при создании объектов. Обычно такие параметры используются для управления способом, которым память должна выделяться для объектов

Например, можно передать параметры, которые исключают случайное прерывание приложения при выделении больших объёмов памяти

43. Дополнительные операции преобразования типов: const_cast, dynamic_cast, static_cast, reinterpret_cast

1. `const_cast` - используется для устранения значения `const` и `volatile` и изменения типа указателя. Это может быть полезно, когда нужно работать с функциями или методами, которые не являются `const`-совместимыми, но которые тем не менее должны работать с значениями `const`
2. `dynamic_cast` - преобразование при выполнении, которое используется для преобразования указателя на базовый класс в указатель на его производный класс
3. `static_cast` - явное преобразование типа, позволяющее преобразовать один тип в другой. Например, можно использовать `static_cast` для конвертации типа `int` в тип `double`
4. `reinterpret_cast` - позволяет преобразовать указатель, которые ссылаются на один тип данных, указатель на другой тип данных. Он выполняет битовый пересчёт значения указателя в другой указатель

44. Понятие о динамической идентификации типов в C++. Операция typeid. Основные функции класса typeid. Примеры использования

Динамическая идентификация типов - это механизм, который позволяет определять тип объекта во время выполнения программы. Она основана на использовании операции `typeid`, которая возвращает объект типа `typeid`, содержащий информацию о типе объекта

Основные функции класса `typeid` включают функцию `name()`, которая вызывается для получения имени типа объекта, и функцию `operator==()`, которая используется для определения, являются ли два объекта типа `typeid` одинаковыми

Пример использования `typeid` заключается в динамическом разрешении виртуальных вызовов методов. Также, `typeid` может быть использован для реализации определённых вариантов обработки ошибок и отладки приложений, например, чтобы определить, является ли объект исключением определённого типа или для проверки типов аргументов при передаче функции